expression that uses node facts and metadata. Tags are used to match nodes and policies.

Policy: This takes care of 'Combining it all' with the use of ordered tables which combine all the above elements in the form of YAML.

Setting up the Razor server

It is recommended that the Razor server shouldn't be installed on the same machine on which the Puppet master is running. The reason is that the default port for Razor is 8080, which conflicts with the default Puppet DB port. To set up a test environment, we will need at least 2 VMs – one for the Puppet master and the other for the Razor server:

- Puppet server (hostname - *puppetmaster*)
- Razor server (hostname - *puppetagent1*)

Razor has been specifically validated on RHEL/CentOS 6.5 but it should work on all 6.x versions. I assume that the Puppet server is installed and configured properly on CentOS 6.5 VM with the hostname *puppetmaster*. If you are new to Puppet, I recommend reading *https://docs.puppetlabs.com/guides/install_puppet/install_el.html*

Here are the steps that need to be followed to set up a Razor server on the *puppetagent1* machine.

Installing a Razor module: A Razor module is available under the Puppet Labs GitHub repository. I assume that the Git package is already installed on *puppetagent1*. We will use Rubygems software (rightly called a 'gem') which allows you to easily download, install and use Ruby packages on the system.

```
   # gem install bundler
 # cd /opt; git clone https://github.com/puppetlabs/razor-server.git
  # cd razor-server;
  # bundle install
  # rake db:migrate
  # torquebox deploy
  #yum install jruby
  #curl -L -O http://torquebox.org/release/org/torquebox/torquebox-dist/3.0.                    1/torquebox-dist-3.0.1-bin.zip
#unzip torquebox-dist-3.0.1-bin.zip -d $HOME
# jruby bin/razor-admin -e production migrate-database
```

Set the following environmental variable:

```
#cat /root/.bashrc
export TORQUEBOX_HOME=$HOME/torquebox-3.0.1
export JBOSS_HOME=$TORQUEBOX_HOME/jboss
export JRUBY_HOME=$TORQUEBOX_HOME/jruby
```

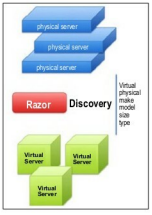

Figure 1: Razor discovering capability

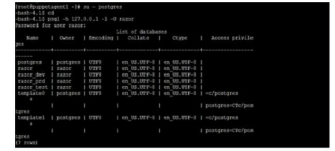

Figure 2: Verifying the Razor database connectivity

```
export PATH=$JRUBY_HOME/bin:$PATH
```

Installing the database: Razor uses PostgreSQL as its database server. To configure the database, follow the steps:

```
 # yum remove postgresql postgresql-server
 # curl -O http://yum.postgresql.org/9.4/redhat/rhel-6-x86_64/pgdg-centos94-9.4-1.noarch.rpm
# rpm -ivh pgdg-centos94-9.4-1.noarch.rpm
# service postgresql-9.4 initdb
# chkconfig postgresql-9.4 on
Log in as psql user and verify the table entry (see Figure 2).
```

Installing the micro-kernel: Download a pre-built micro-kernel from *http://links.puppetlabs.com/razor-microkernel-latest.tar*. The micro-kernel is based on Fedora 19 and needs to be manually put into your *repo_store_root* directory; it cannot be added using the API. If you downloaded the prebuilt micro-kernel above, simply extract it into your *repo_store_root* directory. Doing so will create a sub-directory called *microkernel* with its contents.

```
#cd /var/lib/razor/repo-store
# wget http://links.puppetlabs.com/razor-microkernel-latest.tar
# tar xvf razor-microkernel-latest.tar
#cd microkernel
# ls
 initrd0.img  README  SHA256SUM  SHA256SUM.sig  vmlinuz0
```

Configuring the database: Edit */opt/razor/config.yaml* and change the database URL setting. Once that is done, you can load the Razor database schema into your PostgreSQL database, and finally start the service (see Figure 3).

Ensure that you have the following line: *repo_store_root: /var/lib/razor/repo-store* placed under */opt/razor/config.yaml*. Verify that razor-server service is in a running state:

```
#service razor-server status
razor-server is running (pid 1380)
```

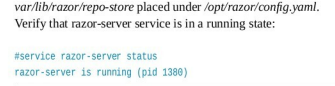

Figure 3: Razor configuration file